BOOK HAVEN

Library Management System

Project type	Java web application
Purpose	Help library staff manage books, readers, borrowing and returns.
Technology	Java 21, Jakarta Servlets, JDBC, embedded Tomcat and H2 database.
Website while running	http://localhost:8080/
Source code	https://github.com/chandraabhinav68-pixel/Library-Management-System

In simple words: Book Haven is a small library desk in a browser. It keeps one shared record of the books and members, and records each book checkout and return.

A beginner-friendly guide to what was built, how the parts work together, and how to run it.

1. What the project does

A library needs to know which books it owns, who its members are, and which books are currently borrowed. Book Haven brings these tasks together in one browser-based application.

The problem and the solution

Paper notebooks and separate spreadsheets can be difficult to search and keep in sync. This application gives staff a single place to look up books, add members, lend available copies, record returns, and see a summary on the dashboard.

Typical staff workflow

Step	Staff action	What is recorded
1	Open the dashboard.	Current book, member, loan and overdue totals.
2	Search for a title or author.	Matching books and copy availability.
3	Register a reader if needed.	Name, email, phone and join date.
4	Issue an available book to a reader.	Borrow date and a due date 14 days later.
5	Record the return.	Return date and updated availability.

Who it is for

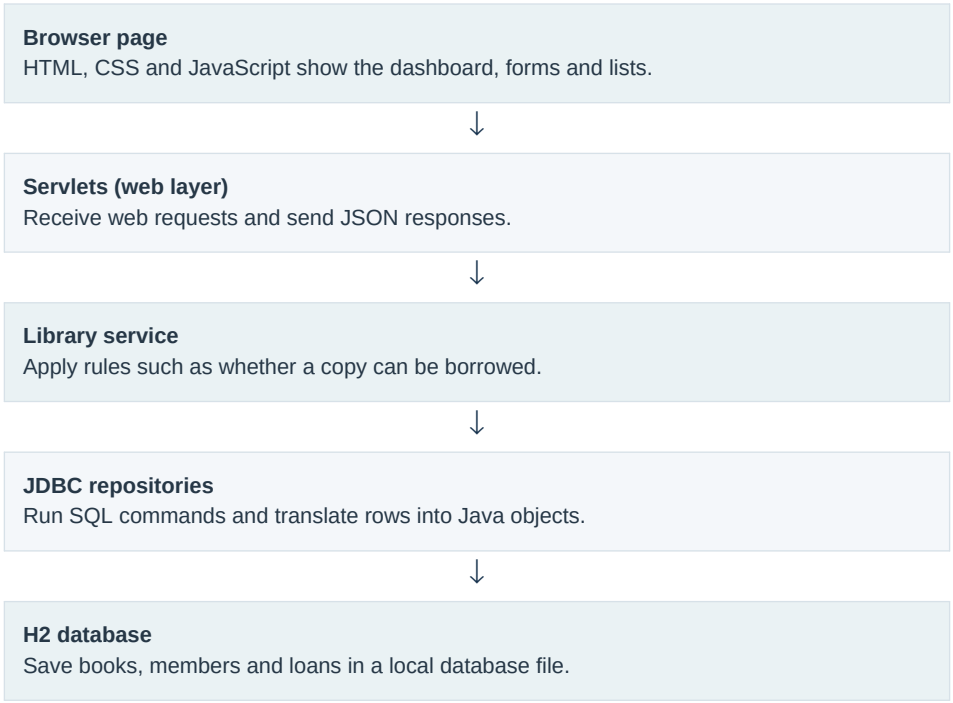
The first version is designed for library staff or a project reviewer demonstrating the circulation workflow. It is a working project foundation; it does not yet include sign-in or different staff roles.

Key outcomes

• One catalog with title, author, ISBN, category and copy count.
• One member register with unique email addresses.
• Loan history that links a book to a member and tracks due/return dates.
• A responsive page that can be used on a desktop or a smaller screen.

2. How the application works

The application is split into layers. Each layer has one main job. The browser sends requests; Java applies library rules; database classes save and read records.



Example: issuing a book

The staff member selects a member and a book. The page sends the request to a servlet. The service checks the library rules. The JDBC repository checks and saves the loan in H2. The response goes back to the page, which refreshes the visible data.

Why use layers?

It makes the code easier to understand and change. For example, SQL is kept in repository classes instead of mixed into the page, and loan rules are handled in a service class instead of duplicated across screens.

3. Project folders and files

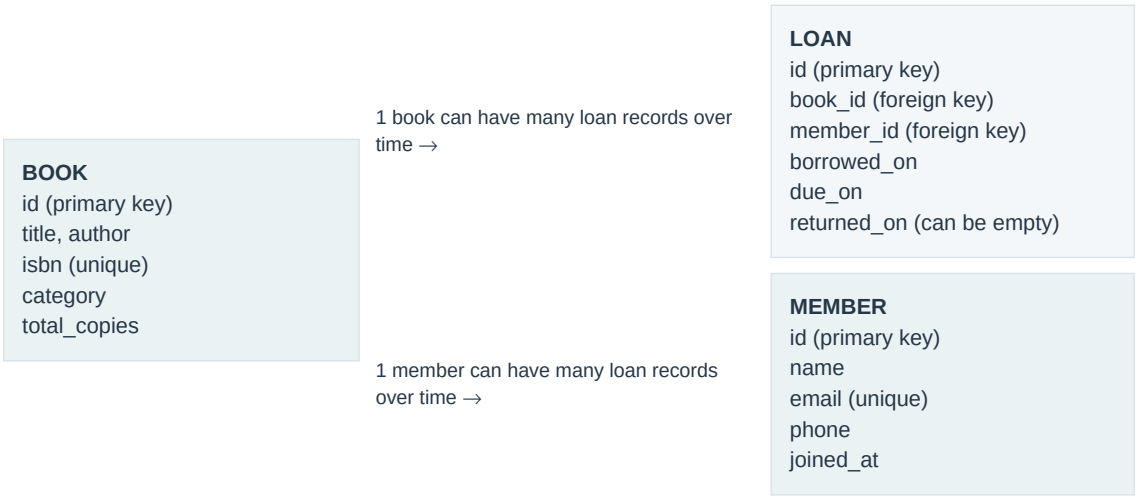
The project is arranged so a reader can find the database, documentation, Java code and website files separately.

Location	What it contains
pom.xml	Maven project settings, dependencies and run configuration.
README.md	Quick project overview, setup, run command, features and API list.
Open Book Haven.url	Shortcut to the local website address: http://localhost:8080/.
database/	SQL schema and starter records for the database.
docs/	Project overview, API notes, database design, screenshots guide and present
src/main/java/com/bookhaven/config/	Database setup, connection settings and application startup.
src/main/java/com/bookhaven/dao/	Repository interfaces and JDBC implementations that read/write data.
src/main/java/com/bookhaven/model/	Java objects for books, members, loans and their shared base entity.
src/main/java/com/bookhaven/service/	Library workflow and the scheduled overdue scan.
src/main/java/com/bookhaven/web/	Servlets that connect web requests with application services.
src/main/webapp/	Website page, styles and browser scripts.
data/	Created automatically on first run; contains the local H2 database file and is n

Where to start reading: Begin with README.md, then docs/PROJECT_OVERVIEW.md for the problem and architecture, and docs/DATABASE_DESIGN.md for the table relationships.

4. Database design

The project uses H2, a small database that runs inside the application and stores its data in a local file. This keeps the demonstration easy to start; a separate database server is not required.



What the links mean

Each loan row points to exactly one book and one member. One book can appear in many loan rows as it is borrowed over time. One member can also have many loan rows. This keeps book and member details from being copied into every loan record.

Important data rules

- ISBN and member email must be unique.
- Foreign keys ensure each loan points to a real book and member.
- An empty returned_on date means the loan is still active.
- Available copies are total copies minus active loans.
- Issue is rejected if no copies are available.
- Due date is 14 days after the borrow date.
- Borrow/return updates use a transaction so related changes are saved together.

Database files

The SQL design is in database/schema.sql and starter examples are in database/seed.sql. On first run, the application creates data/bookhaven-library.mv.db. This generated data folder is excluded from GitHub so the project source stays separate from a machine’s demo records.

5. Main features and how to use them

Start the application, open the dashboard, and follow these everyday library tasks.

Dashboard

Shows summary counts for catalog books, members, loans and overdue items.

Book catalog

Browse and search by text, then add a book with title, author, ISBN, category and total copies.

Member register

Add a member with name, email and phone. Email is checked for duplicates.

Issue a book

Choose a member and a book with availability. The system records the borrow date and sets a two-week due date.

Return a book

Return an active loan. The return date is recorded and the available-copy count changes.

Responsive interface

HTML, CSS and JavaScript provide a single page designed to adapt to desktop and narrow screens.

Clear error handling

Invalid or conflicting requests receive useful errors, for example when a requested copy is unavailable.

HTTP API

The browser communicates with the Java application using JSON over HTTP. Main routes are **GET /api/dashboard**, **GET /api/books?q=...**, **POST /api/books**, **GET/POST /api/members**, **GET/POST /api/loans**, and **POST /api/loans/{id}/return**. More examples are documented in docs/API.md.

6. How the Java work maps to the review rubric

This table points reviewers to the parts of the project that demonstrate each area. The report describes the implemented design and code structure; it does not claim features that are outside the current project.

Rubric area	What is demonstrated	Where to find it
Problem and solution design	A library workflow is described from book search through return.	docs/PROJECT_OVERVIEW.md , docs/DATABASE_DESIGN.md
Core Java and OOP	Shared abstract LibraryEntity; Book, Member and Loan types; repository interfaces; service classes; domain exceptions.	model/, dao/, service/, exception/
Collections and generics	Typed repository interfaces use generic types and return collections such as List.	dao/LibraryRepository.java and related interfaces
Multithreading and synchronization	OverdueScanService schedules a periodic background scan; loan changes use transaction boundaries and availability checks.	service/OverdueScanService.java, JDBC loan repository
Database operation classes	JDBC repository classes separate SQL access for books, members and loans.	dao/Jdbc*Repository.java
JDBC connectivity	Database configuration opens H2 connections; prepared statements execute SQL.	config/Database.java, dao/
Servlet and web integration	Servlets expose JSON routes; embedded Tomcat mounts the app and serves the page.	web/, LibraryApplication.java
UI and responsiveness	Browser assets render the dashboard and circulation forms.	src/main/webapp/, docs/SCREENSHOTS.md

How to present it: Show the dashboard, add/search a book, register a member, issue a copy, then record a return. Explain that the service applies the rules, JDBC repositories store data, and the three linked database tables preserve the history.

Scope note

The current version does not include login/roles, fines, email reminders, barcode scanning or hosting on a public server. These are sensible next features, but are not needed to describe the working local demo.

7. Run the project and open the website

You need a Java Development Kit (JDK) version 21 or newer and Apache Maven 3.9 or newer.

Start it

```
mvn clean compile exec:java
```

When Maven reports that Tomcat has started, open **http://localhost:8080/** in a browser. The direct local shortcut is the file **Open Book Haven.url** in the project folder. Stop the server with **Ctrl+C** in the terminal where it is running.

Share the source code

The project repository is **https://github.com/chandraabhinav68-pixel/Library-Management-System**. The local website address uses localhost, which means it is available on the computer running the server. To let reviewers visit it over the internet, deploy the application to a public Java hosting service and replace the link with that deployed address.

Project quality and next steps

The project has a clear folder layout, setup instructions, database documentation, a presentation, and modular Java code. Before a formal review, run the app on the presentation computer and capture fresh screenshots. A useful next iteration would add staff sign-in and role checks, then deploy a public demo if the review requires remote access.

Quick review checklist

[ ] JDK and Maven available
[ ] Start command completes
[ ] Open http://localhost:8080/
[ ] Demonstrate search, member registration, issue and return
[ ] Show the database design and rubric mapping
[ ] Share the GitHub repository link

Prepared from the project source tree and documentation. Website address is local to the running computer.